

DATABASE

Table 1

Field 1	Field 2	Field 3
Value	Value	Value
Value	Value	Value
Value	Value	Value
Value	Value	Value

Table 2

Field 1	Field 2	Field 3
Value	Value	Value
Value	Value	Value
Value	Value	Value
Value	Value	Value

Table 3

Field 1	Field 2	Field 3
Value	Value	Value
Value	Value	Value
Value	Value	Value
Value	Value	Value

Attributes

So, table is a set of tuples/rows

Table - Student

Student Name

Date of Joining

Course
Name

Contact
Number

Ramesh Chandra

01-Jan-2014

.NET

9999999999

Suresh Jain

03-Jan-2014

.NET

8888888888

Shraddha Negi

03-Jan-2014

JAVA

7777777777

Maulik Mehta

10-Feb-2014

Testing

7777777777

Maulik Mehta

15-Mar-2014

.NET

7777999999

Row/
Tuple

DEFINITION SUMMARY

<u>Informal Terms</u>	<u>Formal Terms</u>
Table	Relation
Column	Attribute/Domain
Row	Tuple
Values in a column	Domain
Table Definition	Schema of a Relation
Populated Table	Extension

SQL Languages



DDL

DML

DQL

DCL

TCL

DAL

- *DDL* (Data Definition Language)
- *DML* (Data Manipulation Language)
- *DQL* (Data Query Language)
- *DCL* (Data Control Language)
- *TCL* (Transactional control Language)
- *DAL* (Data Administration Language)

SQL Commands type

DDL

Create

Alter

Drop

Rename

Truncate

DML

Select

Insert

Update

Delete

DCL

Grant

Revoke

TCL

Rollback

Commit

Data Definition Language DDL

01

Create Table statement

```
CREATE TABLE movies ( Movie_number number, Movie_name varchar2(100),  
Movie_type varchar2(40), Movie_release_date date );
```

02

Add column to table

```
ALTER TABLE movies ADD (movie_language varchar2(30));
```

03

Modify Column attributes

```
ALTER TABLE movies MODIFY (movie_type varchar2(50));
```

04

Drop Table

```
DROP TABLE movies;
```

05

Insert Values into a table

```
INSERT INTO movies VALUES ( 01, 'TERMINATOR', 'ACTION', '12-JAN-2015' );  
COMMIT;
```

06

Update a record

```
UPDATE movies set movie_release_date = '14-jan-2015' WHERE movie_number = 101;  
COMMIT;
```

07

Delete a record

```
DELETE from movies WHERE movie_name = 'RUSH HOUR';  
COMMIT;
```

08

Truncate Statement

```
TRUNCATE TABLE SALES;
```

DDL

Introduction

To understand the SQL Data Definition Language

Create

Alter

Drop

Truncate

Rename

CREATE TABLE

The basic syntax of the CREATE TABLE statement is as follows -

```
CREATE TABLE table_name(  
    column1 datatype,  
    column2 datatype,  
    column3 datatype,  
    .....  
    columnN datatype  
);
```

```
CREATE TABLE CUSTOMERS(  
    ID NUMBER,  
    NAME VARCHAR (20),  
    AGE NUMBER,  
    ADDRESS CHAR (25) ,  
    SALARY NUMBER,  
);
```

Creating a Table

Syntax

```
CREATE TABLE table_name  
(Column_name      datatype[(size)],  
 Column_name      datatype[(size)],  
)
```

Example

```
CREATE TABLE books  
(ISBN             char(20),  
 Title            char(50),  
 AuthorID         Integer,  
 Price            float)
```

Creates a table with four columns

ALTER TABLE

The basic syntax of an ALTER TABLE command to add a **New Column**:

```
ALTER TABLE table_name  
ADD  
column_name datatype;
```

```
ALTER TABLE Customers  
ADD  
Mobile varchar2(10);
```

To **DROP COLUMN** in an existing table:

```
ALTER TABLE table_name  
DROP COLUMN  
column_name ;
```

```
ALTER TABLE Customers  
DROP COLUMN  
Mobile;
```

ALTER TABLE

To **change the DATA TYPE** of a column in a table:

```
ALTER TABLE table_name  
MODIFY  
column_name new_datatype;
```

```
ALTER TABLE Customers  
MODIFY  
Name varchar2(30);
```

To **rename column name** in a table:

```
ALTER TABLE table_name  
RENAME COLUMN  
Old_Column_Name TO New_Column_Name;
```

```
ALTER TABLE Customers  
RENAME COLUMN  
Name TO FullName;
```

ALTER TABLE

To **RENAME TABLE**:

```
ALTER TABLE table_name  
RENAME TO  
New_Table_Name;
```

```
ALTER TABLE Customers  
RENAME TO  
CustomerDetails;
```

DROP TABLE

To **DROP TABLE** :

```
DROP TABLE table_name;
```

```
DROP TABLE Customers;
```

Constraints

Constraints are the rules enforced on the data columns of a table. These are used to limit the type of data that can go into a table. This ensures the accuracy and reliability of the data in the database.

Constraints can be divided into following two types:

- **Column level** constraints : limits only column data
- **Table level** constraints : limits whole table data

1. NOT NULL
2. UNIQUE
3. PRIMARY KEY
4. FOREIGN KEY
5. CHECK
6. DEFAULT

Defining Constraints

- Syntax:

```
CREATE TABLE [schema.]table  
    (column datatype [DEFAULT expr]  
      [column_constraint],  
      ...  
      [table_constraint] [...]);
```

- Column-level constraint syntax:

```
column [CONSTRAINT constraint_name] constraint_type,
```

- Table-level constraint syntax:

```
column, ...  
    [CONSTRAINT constraint_name] constraint_type  
    (column, ...),
```


Defining Constraints

- Example of a column-level constraint:

```
CREATE TABLE employees(  
  employee_id  NUMBER(6)  
    CONSTRAINT emp_emp_id_pk PRIMARY KEY,  
  first_name   VARCHAR2(20),  
  ...);
```

1

- Example of a table-level constraint:

```
CREATE TABLE employees(  
  employee_id  NUMBER(6),  
  first_name   VARCHAR2(20),  
  ...  
  job_id       VARCHAR2(10) NOT NULL,  
  CONSTRAINT emp_emp_id_pk  
    PRIMARY KEY (EMPLOYEE_ID));
```

2

UNIQUE Constr

UNIQUE Constraint

Defined at either the table level or the column level:

EMPLOYEES

UNIQUE constr

	EMPLOYEE_ID	LAST_NAME	EMAIL
1	100	King	SKING
2	101	Kochhar	NKOCHHAR
3	102	De Haan	LDEHAAN
4	103	Hunold	AHUNOLD
5	104	Ernst	BERNST
6	107	Lorentz	DLORENTZ

...



INSERT INTO

208 SMITH	JSMITH
-----------	--------

← Allowed

209 SMITH	JSMITH
-----------	--------

← Not allowed: already exists

```
CREATE TABLE employees(  
    employee_id    NUMBER(6),  
    last_name      VARCHAR2(25) NOT NULL,  
    email          VARCHAR2(25),  
    salary         NUMBER(8,2),  
    commission_pct NUMBER(2,2),  
    hire_date      DATE NOT NULL,  
    ...  
    CONSTRAINT emp_email_uk UNIQUE(email));
```

- Example of a column-level constraint:

```
CREATE TABLE employees(  
  employee_id  NUMBER(6)  
    CONSTRAINT emp_emp_id_pk PRIMARY KEY,  
  first_name   VARCHAR2(20),  
  ...);
```

1

- Example of a table-level constraint:

```
CREATE TABLE employees(  
  employee_id  NUMBER(6),  
  first_name   VARCHAR2(20),  
  ...  
  job_id       VARCHAR2(10) NOT NULL,  
  CONSTRAINT emp_emp_id_pk  
    PRIMARY KEY (EMPLOYEE_ID) );
```

2

DEPARTMENTS

PRIMARY KEY

DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	10 Administration	200	1700
2	20 Marketing	201	1800
3	50 Shipping	124	1500
4	60 IT	103	1400
5	80 Sales	149	2500
6	90 Executive	100	1700
7	110 Accounting	205	1700
8	190 Contracting	(null)	1700

Not allowed
(null value)



INSERT INTO

(null)	Public Accounting	124	2500
	50 Finance	124	1500

Not allowed
(50 already exists)

FOREIGN KEY Constraint

PRIMARY KEY →

DEPARTMENTS

	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	10	Administration	200	1700
2	20	Marketing	201	1800
3	30	Shipping	124	1500
4	60	IT	103	1400
5	80	Sales	149	2500

...

EMPLOYEES

	EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID
1	200	Whalen	10
2	201	Hartstein	20
3	202	Fay	20
4	205	Higgins	110
5	206	Gietz	110

FOREIGN KEY ←

...



INSERT INTO

200	Ford	9
200	Ford	60

Not allowed
(9 does not
exist)

Allowed

FOREIGN KEY Constraint

Defined at either the table level or the column level:

```
CREATE TABLE employees(  
    employee_id      NUMBER(6),  
    last_name        VARCHAR2(25) NOT NULL,  
    email            VARCHAR2(25),  
    salary            NUMBER(8,2),  
    commission_pct   NUMBER(2,2),  
    hire_date        DATE NOT NULL,  
    ...  
    department_id    NUMBER(4),  
    CONSTRAINT emp_dept_fk FOREIGN KEY (department_id)  
        REFERENCES departments(department_id),  
    CONSTRAINT emp_email_uk UNIQUE(email));
```


The foreign key can also be defined at the column level, provided that the constraint is based on a single column. The syntax differs in that the keywords `FOREIGN KEY` do not appear. For example:

```
CREATE TABLE employees
(
...
department_id NUMBER(4) CONSTRAINT emp_deptid_fk
REFERENCES departments(department_id),
...
)
```

FOREIGN KEY Constraint: Keywords

- FOREIGN KEY: Defines the column in the child table at the table-constraint level
- REFERENCES: Identifies the table and column in the parent table
- ON DELETE CASCADE: Deletes the dependent rows in the child table when a row in the parent table is deleted
- ON DELETE SET NULL: Converts dependent foreign key values to null

CREATE TABLE: Example

```
CREATE TABLE teach_emp (  
    empno      NUMBER(5) PRIMARY KEY,  
    ename      VARCHAR2(15) NOT NULL,  
    job        VARCHAR2(10),  
    mgr        NUMBER(5),  
    hiredate   DATE DEFAULT (sysdate),  
    photo      BLOB,  
    sal        NUMBER(7,2),  
    deptno     NUMBER(3) NOT NULL  
    CONSTRAINT admin_dept_fkey REFERENCES  
        departments(department_id));
```

NOT NULL

NOT NULL constraint restricts a column from having a NULL value. Once **NOT NULL** constraint is applied to a column, you cannot pass a null value to that column. It enforces a column to contain a proper value. One important point to note about NOT NULL constraint is that **it cannot be defined at table level**.

```
CREATE TABLE CUSTOMERS(  
  ID INT NOT NULL,  
  NAME VARCHAR (20) NOT NULL,  
  AGE INT NOT NULL,  
  ADDRESS CHAR (25) ,  
  SALARY DECIMAL (18, 2),  
  PRIMARY KEY (ID)  
);
```

```
ALTER TABLE CUSTOMERS  
MODIFY SALARY DECIMAL (18, 2) NOT NULL;
```

CHECK

CHECK constraint is used to restrict the value of a column between a range. It performs check on the values, before storing them into the database. Its like condition checking before saving data into a column

```
CREATE TABLE CUSTOMERS(  
  ID INT NOT NULL,  
  NAME VARCHAR (20) NOT NULL,  
  AGE INT NOT NULL CHECK (AGE >= 18),  
  ADDRESS CHAR (25) ,  
  SALARY DECIMAL (18, 2),  
  PRIMARY KEY (ID)  
);
```

```
ALTER TABLE CUSTOMERS  
MODIFY AGE INT NOT NULL CHECK (AGE >= 18 );
```

```
ALTER TABLE CUSTOMERS  
ADD CONSTRAINT myCheckConstraint CHECK(AGE >= 18);
```

UNIQUE

UNIQUE constraint ensures that a field or column will only have unique values. A UNIQUE constraint field will not have duplicate data. UNIQUE constraint **can be applied at column level or table level.**

```
CREATE TABLE CUSTOMERS(  
  ID INT NOT NULL,  
  NAME VARCHAR (20) NOT NULL,  
  AGE INT NOT NULL UNIQUE,  
  ADDRESS CHAR (25) ,  
  SALARY DECIMAL (18, 2),  
  PRIMARY KEY (ID)  
);
```

```
ALTER TABLE CUSTOMERS  
  MODIFY AGE INT NOT NULL UNIQUE;
```

```
ALTER TABLE CUSTOMERS  
  ADD CONSTRAINT myUniqueConstraint UNIQUE(AGE, SALARY);
```

```
ALTER TABLE CUSTOMERS  
  DROP CONSTRAINT myUniqueConstraint;
```

PRIMARY KEY

PRIMARY KEY constraint uniquely identifies each record in a database. A Primary Key must contain unique value and it must not contain null value. Usually Primary Key is used to index the data inside the table.

```
CREATE TABLE CUSTOMERS(  
  ID INT NOT NULL,  
  NAME VARCHAR (20) NOT NULL,  
  AGE INT NOT NULL,  
  ADDRESS CHAR (25) ,  
  SALARY DECIMAL (18, 2),  
  PRIMARY KEY (ID)  
);
```

```
ALTER TABLE CUSTOMER ADD PRIMARY KEY (ID);
```

```
ALTER TABLE CUSTOMERS  
ADD CONSTRAINT PK_CUSTID PRIMARY KEY (ID, NAME);
```

```
ALTER TABLE CUSTOMERS DROP PRIMARY KEY ;
```

FOREIGN KEY

FOREIGN KEY is used to relate two tables. FOREIGN KEY constraint is also used to restrict actions that would destroy links between tables

```
CREATE TABLE CUSTOMERS(  
  ID INT NOT NULL,  
  NAME VARCHAR (20) NOT NULL,  
  AGE INT NOT NULL,  
  ADDRESS CHAR (25) ,  
  SALARY DECIMAL (18, 2),  
  PRIMARY KEY (ID)  
);
```

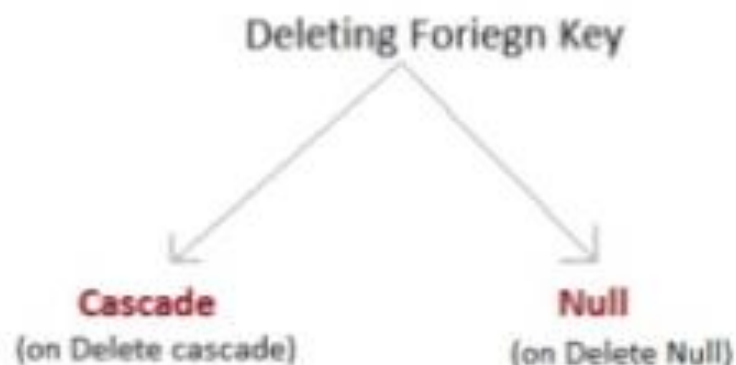
```
CREATE TABLE ORDERS (  
  ID INT NOT NULL,  
  DATE DATETIME,  
  CUSTOMER_ID INT references CUSTOMERS(ID),  
  AMOUNT double,  
  PRIMARY KEY (ID)  
);
```

```
ALTER TABLE ORDERS  
  ADD FOREIGN KEY (Customer_ID) REFERENCES CUSTOMERS (ID);
```

```
ALTER TABLE ORDERS  
  DROP FOREIGN KEY;
```

FOREIGN KEY

There are two ways to maintain the integrity of data in Child table, when a particular record is deleted in main table. When two tables are connected with Foreign key, and certain data in the main table is deleted, for which record exist in child table too, then we must have some mechanism to save the integrity of data in child table.



- **On Delete Cascade** : This will remove the record from child table, if that value of foreign key is deleted from the main table.
- **On Delete Null** : This will set all the values in that record of child table as NULL, for which the value of foreign key is deleted from the main table.

Examine the Violations

COMPOSITE UNIQUE KEY VIOLATION

Composite keys must be defined at the table level.

```
CREATE TABLE clients(  
  client_number  NUMBER(6),  
  first_name     VARCHAR2(20),  
  last_name      VARCHAR2(20),  
  phone          VARCHAR2(20) CONSTRAINT phone_email_uk  
                UNIQUE(email,phone),  
  email          VARCHAR2(10) CONSTRAINT NOT NULL,  
  CONSTRAINT emailclients_email NOT NULL,  
  CONSTRAINT clients_client_num_pk PRIMARY KEY (client_number));
```

NOT NULL VIOLATION

NOT NULL constraints can only be defined at the column level.

NAME VIOLATION

When using the term CONSTRAINT, it must be followed by a constraint name.

CHECK

CHECK constraint is used to restrict the value of a column between a range. It performs check on the values, before storing them into the database. Its like condition checking before saving data into a column

```
CREATE TABLE CUSTOMERS(  
  ID INT NOT NULL,  
  NAME VARCHAR (20) NOT NULL,  
  AGE INT NOT NULL CHECK (AGE >= 18),  
  ADDRESS CHAR (25) ,  
  SALARY DECIMAL (18, 2),  
  PRIMARY KEY (ID)  
);
```

```
ALTER TABLE CUSTOMERS  
MODIFY AGE INT NOT NULL CHECK (AGE >= 18 );
```

```
ALTER TABLE CUSTOMERS  
ADD CONSTRAINT myCheckConstraint CHECK(AGE >= 18);
```

DDL

Specifying Keys- Multiple Columns

Specifying multiple columns as unique:

Example:

```
CREATE TABLE Studios
(studio_id      Number,
name           char(20),
city           varchar(50),
state          char(2),
UNIQUE (name),
UNIQUE(city, state))
```

Here both name & city/state combination are declared as candidate keys

DDL

Specifying Keys- Primary Key

- Specifying multiple columns as unique:
- To specify the Primary Key the **Primary Key** clause is used

- **Example:**

```
CREATE TABLE Studios
(studio_id      Number,
name           char(20),
city           varchar(50),
state          char(2),
PRIMARY KEY (studio_id),
UNIQUE (name),
UNIQUE(city, state)
)
```

DDL

Specifying Keys- Single and MultiColumn Keys

- Single column keys can be defined at the column level instead of at the table level at the end of the field descriptions.
- MultiColumn keys still need to be defined separately at the table level

```
CREATE TABLE Studios
(studio_id      Number      PRIMARY KEY,
name           char(20)     UNIQUE,
city           varchar(50),
state          char(2),
Unique(city, state))
```

- Note: Some databases require the use of Unique Index for specification of keys.

DDL

Specifying Keys- Foreign Keys

- References clause is used to create a relationship between a set of columns in one table and a candidate key in the table that is being referenced.

- **Example:**

```
CREATE TABLE Movies
(movie_title      varchar(40),
studio_id        Number REFERENCES Studios(studio_id))
```

- Creates a relationship from the Movies table to the Studios table

DDL

Constraints- Disallowing Null Values

Disallowing Null Values:

- Null values entered into a column means that the data is not known.
 - These can cause problems in Querying the database.
 - Specifying Primary Key automatically prevents null being entered in columns which specify the primary key
-
- Not Null clause is used in preventing null values from being entered in a column.

- **Example:**

```
CREATE TABLE Studios
( studio_id      number           PRIMARY KEY,
  name           char(20)         NOT NULL,
  city           varchar(50)      NOT NULL,
  state          char(2)          NOT NULL)
```

- Null clause can be used to explicitly allow null values in a column also

DDL

Constraints- Value Constraints

Value Constraints:

- Allows value inserted in the column to be checked condition in the column constraint.
- Check clause is used to create a constraint in SQL

- **Example:**

```
CREATE TABLE Movies
(movie_title      varchar(40)          PRIMARY KEY,
 studio_id       Number,
 budget          Number check (budget > 50000)
)
```

- Table level constraints can also be defined using the Constraint keyword

- **Example:**

```
CREATE TABLE Movies
(movie_title      varchar(40)          PRIMARY KEY,
 studio_id       Number,
 budget          Number check (budget > 50000),
 release_date     Date,
 CONSTRAINT release_date_constraint Check (release_date between '01-Jan-1980' and '31-dec-1989'))
```

- Such constraints can be activated and deactivated as required.

DDL

Constraints- Default Value

Default Value:

- A default value can be inserted in any column by using the Default keyword.

- **Example:**

```
CREATE TABLE Movies (  
  movie_title      varchar(40)      NOT NULL,  
  release_date     date              DEFAULT sysdate NULL,  
  genre            varchar(20)      DEFAULT 'Comedy'      Check genere  
    In ('Comedy', 'Horror', 'Drama')  
)
```

- Table level constraints can also be defined using the Constraint keyword
- release_date defaults to the current date, however Null value is enabled in the column which will need to be added explicitly when data is added.
- **Note:** Any valid expression can be used while specifying constraints

1. CREATE TABLE Professors (
 prof_ssn CHAR(10),
 name CHAR(64),
 age INTEGER,
 rank INTEGER,
 speciality CHAR(64),
 PRIMARY KEY (prof_ssn))
2. CREATE TABLE Depts (
 dno INTEGER,
 dname CHAR(64),
 office CHAR(10),
 PRIMARY KEY (dno))
3. CREATE TABLE Runs (
 dno INTEGER,
 prof_ssn CHAR(10),
 PRIMARY KEY (dno, prof_ssn),
 FOREIGN KEY (prof_ssn) REFERENCES Professors,
 FOREIGN KEY (dno) REFERENCES Depts)
4. CREATE TABLE Work_Dept (
 dno INTEGER,
 prof_ssn CHAR(10),
 pc_time INTEGER,
 PRIMARY KEY (dno, prof_ssn),
 FOREIGN KEY (prof_ssn) REFERENCES Professors,
 FOREIGN KEY (dno) REFERENCES Depts)

INSERT

The basic syntax of INSERT:

```
INSERT INTO TABLE_NAME (column1, column2, column3,...columnN)  
VALUES (value1, value2, value3,...valueN);
```

```
INSERT INTO Customers VALUES (101, 'Ramesh', 23,'New Delhi, India',10000);
```

```
INSERT INTO Customers (ID, NAME, AGE, ADDRESS, SALARY)
```

```
VALUES (101, 'John', 23,'New Street, UK',10000);
```

```
INSERT INTO TABLE_NAME (column1, column2, column3,...columnN)  
SELECT column1, column2, column3 FROM tablename;
```

```
INSERT INTO EMP_HISTORY
```

```
SELECT EMPLOYEE_ID, EMPLOYEE_NAME, SALARY, DEPARTMENT_ID  
FROM employees;
```

DELETE

The basic syntax of DELETE:

```
DELETE FROM TABLE_NAME [WHERE];
```

```
DELETE FROM Customers WHERE ID=100;
```

```
DELETE FROM Customers;
```

UPDATE

The basic syntax of UPDATE:

```
UPDATE TableName
SET
Column1=Value1,
Column1=Value1,
...,
ColumnN=ValueN
[WHERE];
```

```
UPDATE Customers SET salary =20000
WHERE ID=100;
```

Let's Limit The Data

01

Use of Where clause for filtering numeric value

- `SELECT * FROM sales WHERE total_amount > 1000;`
- `SELECT * FROM sales WHERE total_amount != 44;`
- `SELECT * FROM sales WHERE total_amount^44;`
- `SELECT * FROM sales WHERE quantity <= 10;`

02

Use of Where clause for filtering text value

- `SELECT * FROM sales WHERE sales_date = '09-feb-2015';`
- `SELECT * FROM product WHERE color = 'RED';`

03

Use of Where clause for comparing column values

- `SELECT * FROM sales WHERE total_amount > sales_amount;`

Comparison Conditions

Operator	Meaning
=	Equal to
>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or equal to
<>	Not equal to
BETWEEN ...AND...	Between two values (inclusive)
IN(set)	Match any of a list of values
LIKE	Match a character pattern
IS NULL	Is a null value

Note: The symbols $\neq$ and $\neq$ can also represent the *not equal to* condition

Logical Operators

01

Use of BETWEEN and NOT BETWEEN

- SELECT * FROM sales WHERE total_amount NOT BETWEEN 1 and 100;
- SELECT * FROM sales WHERE total_amount BETWEEN 1 and 100;

02

Use of IN

- SELECT * FROM sales WHERE quantity IN (20,2,10);

03

Use of LIKE

- SELECT * FROM product WHERE product_name LIKE 'Mob%';
- SELECT * FROM product WHERE product_name LIKE '%Mob';
- SELECT * FROM product WHERE product_name LIKE 'Mob_Device';

Logical Operators

04

Use of ALL

- `SELECT * FROM sales WHERE total_amount > ALL (50,100,200);`

05

Use of ANY

- `SELECT * FROM sales WHERE total_amount > ANY (50,100,200);`

06

Use of NULL

- `SELECT * FROM product WHERE color IS NULL;`

07

Use of AND

- `SELECT * FROM sales WHERE total_amount > 100 AND quantity < 20;`

Arithmetic Operator

1) Addition (+) : `SELECT 100/ 20 FROM DUAL; --5`

2) Subtraction (-) : `SELECT 100+20 FROM DUAL; --120`

3) Multiplication (*) : `SELECT 100-20 FROM DUAL; --80`

4) Division (/) : `SELECT 100*20 FROM DUAL; --2000`

5) Modulus (%) : `SELECT 10%100 FROM DUAL; --10`

Let's Sort the Data

Order By Clause

- `SELECT sales_date, product_id, order_id, sales_amount, tax_amount FROM sales ORDER BY tax_amount;`
- `SELECT sales_date, product_id, order_id, sales_amount, tax_amount FROM sales ORDER BY sales_amount, tax_amount;`
- `SELECT order_id, sales_date, product_id, sales_amount, tax_amount FROM sales ORDER BY order_id DESC;`

How NULL values are treated while sorting the Data?

NULL values are treated as very large value by Oracle. So NULL data will sort to the bottom of the sort in ascending order and to the top of the sort in descending order.

Character-Manipulation Functions

- These functions manipulate character strings:

Function	Result
<code>CONCAT('Hello', 'World')</code>	HelloWorld
<code>SUBSTR('HelloWorld', 1, 5)</code>	Hello
<code>LENGTH('HelloWorld')</code>	10
<code>INSTR('HelloWorld', 'W')</code>	6
<code>LPAD(salary, 10, '*')</code>	*****24000
<code>RPAD(salary, 10, '*')</code>	24000*****
<code>REPLACE('JACK and JUE', 'J', 'BL')</code>	BLACK and BLUE
<code>TRIM('H' FROM 'HelloWorld')</code>	elloWorld

Date Functions

Function	Result
MONTHS_BETWEEN	Number of months between two dates
ADD_MONTHS	Add calendar months to date
NEXT_DAY	Next day of the date specified
LAST_DAY	Last day of the month
ROUND	Round date
TRUNC	Truncate date

MONTHS_BETWEEN(*date1*, *date2*): Finds the number of months between *date1* and *date2*. The result can be positive or negative. If *date1* is later than *date2*, the result is positive; if *date1* is earlier than *date2*, the result is negative.

Using Date Functions

Function	Result
MONTHS_BETWEEN ('01-SEP-95' , '11-JAN-94')	19.6774194
ADD_MONTHS ('11-JAN-94' , 6)	'11-JUL-94'
NEXT_DAY ('01-SEP-95' , 'FRIDAY')	'08-SEP-95'
LAST_DAY ('01-FEB-95')	'28-FEB-95'

INPUT:

```
SQL> SELECT SYSDATE, TO_CHAR(SYSDATE, 'DDSPTH MONTH YEAR' )  
FROM DUAL;
```

OUTPUT:

```
SYSDATE TO_CHAR(SYSDATE, 'DDSPTHMONTHYEAR')
```

```
24-MAR-07 TWENTY-FOURTH MARCH TWO THOUSAND SEVEN
```

ANALYSIS:

DD gives the day number

DDSP Writes day number in words

TH is the format. Depends upon the number it gives either ST / RD / ST / ND
format

INPUT:

```
SQL> SELECT TO_CHAR(SYSDATE, 'HH:MI:SS AM') FROM DUAL;
```

OUTPUT:

```
TO_CHAR(SYS
```

```
-----
```

```
08:40:17 PM
```

ANALYSIS:

HH	returns Hours	}	
MI	returns Minutes	}	Returns time from current date
SS	returns Seconds	}	
AM	returns AM / PM		depends on Time

Using Date Functions

Function	Result
MONTHS_BETWEEN ('01-SEP-95' , '11-JAN-94')	19.6774194
ADD_MONTHS ('11-JAN-94' , 6)	'11-JUL-94'
NEXT_DAY ('01-SEP-95' , 'FRIDAY')	'08-SEP-95'
LAST_DAY ('01-FEB-95')	'28-FEB-95'

Using Date Functions

Function	Result
MONTHS_BETWEEN ('01-SEP-95' , '11-JAN-94')	19.6774194
ADD_MONTHS ('11-JAN-94' , 6)	'11-JUL-94'
NEXT_DAY ('01-SEP-95' , 'FRIDAY')	'08-SEP-95'
LAST_DAY ('01-FEB-95')	'28-FEB-95'

```
SELECT column, group_function  
FROM table  
[WHERE condition]  
[GROUP BY group_by_expression]  
[HAVING group_condition]  
[ORDER BY column];
```

EMPLOYEE_ID	FIRST_NAME	LAST_NAME		SALARY		DEPARTMENT_ID
100	Steven	King	—	24000	—	90
101	Neena	Kochhar	—	17000	—	90
102	Lex	De Haan	—	17000	—	90
103	Alexander	Hunold	—	9000	—	60
104	Bruce	Ernst	—	6000	—	60
105	David	Austin	—	4800	—	60
—	—	—	—	—	—	—
132	TJ	Olson	—	2100	—	50
—	—	—	—	—	—	—

SELECT MIN(salary)
FROM employees;

SALARY
24000
17000
17000
9000
6000
4800
—
2100
—

MIN(salary)
2100

Minimum Salary

EMPLOYEE_ID	FIRST_NAME	LAST_NAME		JOB_ID	SALARY		DEPARTMENT_ID
100	Steven	King	—	AD_PRES	24000	—	90
101	Neena	Kochhar	—	AD_VP	17000	—	90
102	Lex	De Haan	—	AD_VP	17000	—	90
103	Alexander	Hunold	—	IT_PROG	9000	—	60
104	Bruce	Ernst	—	IT_PROG	6000	—	60
105	David	Austin	—	IT_PROG	4800	—	60
106	Valli	Pataballa	—	IT_PROG	4800	—	60
107	Diana	Lorentz	—	IT_PROG	4200	—	60
108	Nancy	Greenberg	—	FI_MGR	12000	—	100
—	—	—	—	—	—	—	—

```

SELECT MAX(salary)
FROM employees
WHERE job_id = 'IT_PROG' ;

```

JOB_ID	SALARY
AD_PRES	24000
AD_VP	17000
AD_VP	17000
IT_PROG	9000
IT_PROG	6000
IT_PROG	4800
IT_PROG	4800
IT_PROG	4200
FI_MGR	12000
—	—

MAX(salary)
9000

maximum salary

EMPLOYEE_ID	FIRST_NAME	LAST_NAME		SALARY		DEPARTMENT_ID
100	Steven	King	-	24000	-	90
101	Neena	Kochhar	-	17000	-	90
102	Lex	De Haan	-	17000	-	90
103	Alexander	Hunold	-	9000	-	60
104	Bruce	Ernst	-	6000	-	60
105	David	Austin	-	4800	-	60
106	Valli	Pataballa	-	4800	-	60
107	Diana	Lorentz	-	4200	-	60
108	Nancy	Greenberg	-	12000	-	100
-	-	-	-	-	-	-

```

SELECT AVG(salary),count(*)
FROM employees
WHERE department_id = 90 ;

```

Average salary and number of employees
working in the department ID 90

SALARY	DEPARTMENT_ID
24000	90
17000	90
17000	90
9000	60
6000	60
4800	60
4800	60
4200	60
12000	100
-	-

1
2
3

AVG(salary)	count(*)
19333.33333	3

EMPLOYEE_ID	FIRST_NAME	LAST_NAME		SALARY		DEPARTMENT_ID
100	Steven	King	—	24000	—	90
101	Neena	Kochhar	—	17000	—	90
102	Lex	De Haan	—	17000	—	90
103	Alexander	Hunold	—	9000	—	60
104	Bruce	Ernst	—	6000	—	60
105	David	Austin	—	4800	—	60
106	Valli	Pataballa	—	4800	—	60
107	Diana	Lorentz	—	4200	—	60
108	Nancy	Greenberg	—	12000	—	100
—	—	—	—	—	—	—

```
SELECT AVG(salary),count(*)
FROM employees
WHERE department_id = 90 ;
```

Average salary and number of employees
working in the department ID 90

SALARY	DEPARTMENT_ID
24000	90
17000	90
17000	90
9000	60
6000	60
4800	60
4800	60
4200	60
12000	100
—	—

1
2
3

AVG(salary)	count(*)
19333.33333	3

EMPLOYEE_ID	FIRST_NAME	LAST_NAME		SALARY		DEPARTMENT_ID
100	Steven	King		24000	—	90
101	Neena	Kochhar		17000	—	90
102	Lex	De Haan		17000	—	90
103	Alexander	Hunold		9000	—	60
104	Bruce	Ernst		6000	—	60
105	David	Austin		4800	—	60
106	Valli	Pataballa		4800	—	60
107	Diana	Lorentz		4200	—	60
108	Nancy	Greenberg		12000	—	100
—	—	—	—	—	—	—

```

SELECT department_id, SUM(salary)
FROM employees
GROUP BY department_id ;

```

DEPARTMENT_ID	SALARY
90	24000
90	17000
90	17000
60	9000
60	6000
60	4800
60	4800
60	4200

58000

department_id	SUM(salary)
0	7000
10	4400
20	19000
30	24900
40	6500
50	156400
60	28800
70	10000
80	304500
90	58000
100	51600

EMPLOYEE_ID	FIRST_NAME	LAST_NAME		JOB_ID	SALARY		DEPARTMENT_ID
100	Steven	King	—	AD_PRES	24000	—	90
101	Neena	Kochhar	—	AD_VP	17000	—	90
102	Lex	De Haan	—	AD_VP	17000	—	90
103	Alexander	Hunold	—	IT_PROG	9000	—	60
104	Bruce	Ernst	—	IT_PROG	6000	—	60
105	David	Austin	—	IT_PROG	4800	—	60
106	Valli	Pataballa	—	IT_PROG	4800	—	60
107	Diana	Lorentz	—	IT_PROG	4200	—	60
108	Nancy	Greenberg	—	FI_MGR	12000	—	100
—	—	—	—	—	—	—	—

```

SELECT job_id, AVG(salary)
FROM employees
WHERE job_id <> 'IT_PROG'
GROUP BY job_id ;

```

job_id have been arranged
in a group

JOB_ID	SALARY
AD_PRES	24000
AD_VP	17000
AD_VP	17000
IT_PROG	9000
IT_PROG	6000
IT_PROG	4800
IT_PROG	4800
IT_PROG	4200
FI_MGR	12000

job_id	AVG(salary)
AC_ACCOUNT	8300
AC_MGR	12000
AD_ASST	4400
AD_PRES	24000
AD_VP	17000
FI_ACCOUNT	7920
FI_MGR	12000
HR_REP	6500

EMPLOYEE_ID	FIRST_NAME	LAST_NAME		JOB_ID	SALARY		DEPARTMENT_ID
100	Steven	King	—	AD_PRES	24000	—	90
101	Neena	Kochhar	—	AD_VP	17000	—	90
102	Lex	De Haan	—	AD_VP	17000	—	90
103	Alexander	Hunold	—	IT_PROG	9000	—	60
104	Bruce	Ernst	—	IT_PROG	6000	—	60
105	David	Austin	—	IT_PROG	4800	—	60
106	Valli	Pataballa	—	IT_PROG	4800	—	60
107	Diana	Lorentz	—	IT_PROG	4200	—	60
108	Nancy	Greenberg	—	FI_MGR	12000	—	100
—	—	—	—	—	—	—	—

```

SELECT job_id, SUM(salary), AVG(salary),
MAX(salary), MIN(salary)
FROM employees
WHERE department_id = '90'
GROUP BY job_id ;

```

job have been arranged in a group
for department_id 90

JOB_ID	SALARY	DEPARTMENT_ID
AD_PRES	24000	90
AD_VP	17000	90
AD_VP	17000	90
IT_PROG	9000	60

job_id	SUM(salary)	AVG(salary)	MAX(salary)	MIN(salary)
AD_PRES	24000	24000	24000	24000
AD_VP	34000	17000	17000	17000

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	—	JOB_ID	—	DEPARTMENT_ID
100	Steven	King	—	AD_PRES	—	90
101	Neena	Kochhar	—	AD_VP	—	90
102	Lex	De Haan	—	AD_VP	—	90
103	Alexander	Hunold	—	IT_PROG	—	60
104	Bruce	Ernst	—	IT_PROG	—	60
105	David	Austin	—	IT_PROG	—	60
106	Valli	Pataballa	—	IT_PROG	—	60
107	Diana	Lorentz	—	IT_PROG	—	60
108	Nancy	Greenberg	—	FI_MGR	—	100

```

SELECT job_id, COUNT(*)
FROM employees
GROUP BY job_id;

```

job_id have been
arranged as a group

job_id
AD_PRES
AD_VP
AD_VP
IT_PROG
IT_PROG
IT_PROG
IT_PROG
IT_PROG

1

2

5

job_id	COUNT(*)
AC_ACCOUNT	1
AC_MGR	1
AD_ASST	1
AD_PRES	1
AD_VP	2
FI_ACCOUNT	5
FI_MGR	1
HR_REP	1
IT_PROG	5
MK_MAN	1
MK_REP	1
PR_REP	1
PU_CLERK	5

EMPLOYEE_ID	FIRST_NAME	LAST_NAME		SALARY		DEPARTMENT_ID
100	Steven	King	—	24000	—	90
101	Neena	Kochhar	—	17000	—	90
102	Lex	De Haan	—	17000	—	90
103	Alexander	Hunold	—	9000	—	60
104	Bruce	Ernst	—	6000	—	60
105	David	Austin	—	4800	—	60
106	Valli	Pataballa	—	4800	—	60
—	—	—	—	—	—	—
132	TJ	Olson	—	2100	—	50
—	—	—	—	—	—	—

SELECT MAX(salary) - MIN(salary) DIFFERENCE
FROM employees;

salary
24000
17000
17000
9000

2100

→ Maximum Salary

→ Minimum Salary

DIFFERENCE
21900

Restricting Group Results with the HAVING Clause

When you use the `HAVING` clause, the Oracle server restricts groups as follows:

1. Rows are grouped.
2. The group function is applied.
3. Groups matching the `HAVING` clause are displayed.

```
SELECT    column, group_function
FROM      table
[WHERE    condition]
[GROUP BY group_by_expression]
[HAVING   group_condition]
[ORDER BY column];
```

Using the HAVING Clause

```
SELECT department_id, MAX(salary)
FROM employees
GROUP BY department_id
HAVING MAX(salary)>10000 ;
```

	DEPARTMENT_ID	MAX(SALARY)
1	90	24000
2	20	13000
3	110	12000
4	80	11000

Restricting Group Results

EMPLOYEES

	DEPARTMENT_ID	SALARY
1	10	4400
2	20	13000
3	20	6000
4	50	5800
5	50	2500
6	50	2600
7	50	3100
8	50	3500
9	60	4200
10	60	6000
11	60	9000
12	80	11000
13	80	10500
14	80	8600

...

18	110	8300
19	110	12000
20	(null)	7000

The maximum salary per department when it is greater than \$10,000

	DEPARTMENT_ID	MAX(SALARY)
1	20	13000
2	80	11000
3	90	24000
4	110	12000

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	JOB_ID	SALARY	DEPARTMENT_ID
100	Steven	King	AD_PRES	24000	90
101	Neena	Kochhar	AD_VP	17000	90
102	Lex	De Haan	AD_VP	17000	90
103	Alexander	Hunold	IT_PROG	9000	60
104	Bruce	Ernst	IT_PROG	6000	60
105	David	Austin	IT_PROG	4800	60
106	Valli	Pataballa	IT_PROG	4800	60
107	Diana	Lorentz	IT_PROG	4200	60
108	Nancy	Greenberg	FI_MGR	12000	100
—	—	—	—	—	—

```

SELECT job_id, MAX(salary)
FROM employees
GROUP BY job_id
HAVING MAX(salary) >=4000 ;

```

job_id have been arranged
in a group

JOB_ID	SALARY
AD_PRES	24000
AD_VP	17000
AD_VP	17000
IT_PROG	9000
IT_PROG	6000
IT_PROG	4800
IT_PROG	4800
IT_PROG	4200
FI_MGR	12000

each are
≥4000

job_id	MAX(salary)
AC_ACCOUNT	8300
AC_MGR	12000
AD_ASST	4400
AD_PRES	24000
AD_VP	17000
FI_ACCOUNT	9000
FI_MGR	12000

EMPLOYEE_ID	FIRST_NAME	LAST_NAME		JOB_ID	SALARY		DEPARTMENT_ID
100	Steven	King	—	AD_PRES	24000	—	90
101	Neena	Kochhar	—	AD_VP	17000	—	90
—	—	—	—	—	—	—	—
120	Matthew	Weiss	—	ST_MAN	8000	—	50
121	Adam	Fripp	—	ST_MAN	8200	—	50
122	Payam	Kaufling	—	ST_MAN	7900	—	50
123	Shanta	Vollman	—	ST_MAN	6500	—	50
124	Kevin	Mourgos	—	ST_MAN	5800	—	50
—	—	—	—	—	—	—	—

```

SELECT department_id, AVG(salary), COUNT(*)
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 10;

```

department_id have been arranged
in a group with average salary
and number of employees in each group

department_id	avg(salary)	count(*)
0	7000.00	1
10	4400.00	1
20	9500.00	2
3	4150.00	6
50	3475.555556	45
50	3475.55	45
60	5760.00	5
70	10000.00	1
80	8955.88235	34
90	19333.33	3
100	8600.00	6
110	10150.00	2

having count(*) > 10

department_id	avg(salary)	count(*)
50	3475.55	45
80	8955.88	34

SQL> select * from emp;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	17-DEC-80	800		20
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7934	MILLER	CLERK	7782	23-JAN-82	1300		10

14 rows selected.

SQL>

```
SQL> select deptno, sum(sal)
  2   from emp
  3   group by deptno;
```

DEPTNO	SUM(SAL)
30	9400
20	10875
10	8750

```
SQL> select deptno, sum(sal)
  2   from emp
  3   group by deptno
  4   having sum(sal)>9000;
```

DEPTNO	SUM(SAL)
30	9400
20	10875

```
SQL> _
```

Aggregate Functions

- `SUM( [ALL | DISTINCT] expression )`
- `AVG( [ALL | DISTINCT] expression )`
- `COUNT( [ALL | DISTINCT] expression )`
- `COUNT(*)`
- `MAX(expression)`
- `MIN(expression)`

Sum

SUM(ALL/DISTINCT expr)

Returns the sum value of expr.

```
SELECT  
  SUM(salary),  
  SUM(ALL salary),  
  SUM(DISTINCT salary)  
FROM  
  employees;
```

⚡ SUM(SALARY)	⚡ SUM(ALLSALARY)	⚡ SUM(DISTINCTSALARY)
691416	691416	409908

Avg

AVG(ALL/DISTINCT expr)

Returns the average value of expr.

```
SELECT
  AVG(salary),
  AVG(ALL salary),
  AVG(DISTINCT salary)
FROM
  employees;
```

AVG(SALARY)	AVG(ALLSALARY)	AVG(DISTINCTSALARY)
6461.831775700934579439252336448598130841	6461.831775700934579439252336448598130841	7067.37931034482758620689651

Count

COUNT(*) OR **COUNT(ALL/DISTINCT expr)**

Returns the number of rows in the query. If you specify *expr* then count ignore nulls. If you specify the asterisk (*), this function returns all rows, including duplicates and nulls. COUNT never returns null.

```
SELECT
  COUNT(manager_id),
  COUNT(*),
  COUNT(ALL manager_id),
  COUNT(DISTINCT manager_id)
FROM
  employees;
```

⚙ COUNT(MANAGER_ID)	⚙ COUNT(*)	⚙ COUNT(ALLMANAGER_ID)	⚙ COUNT(DISTINCTMANAGER_ID)
106	107	106	18

Max

MAX(ALL/DISTINCT expr)

Returns the maximum value of expr.

SELECT

MAX(salary),

MAX(**ALL** salary),

MAX(**DISTINCT** salary)

FROM

employees;

MAX(SALARY)	MAX(ALLSALARY)	MAX(DISTINCTSALARY)
24000	24000	24000

Aggregate Functions

- A WHERE clause is used to filter rows BEFORE the GROUPING action
- A HAVING clause filters rows AFTER the GROUPING action

```
SELECT
    department_id,
    SUM(salary)
FROM
    employees
WHERE
    department_id <> 100
GROUP BY
    department_id
HAVING
    SUM(salary) >= 15000;
```


Employee

EmployeeID	Ename	DeptID	Salary
1001	John	2	4000
1002	Anna	1	3500
1003	James	1	2500
1004	David	2	5000
1005	Mark	2	3000
1006	Steve	3	4500
1007	Alice	3	3500

SELECT DeptID, AVG(Salary)
FROM Employee
GROUP BY DeptID;

GROUP BY
Employee Table
using DeptID

DeptID	AVG(Salary)
1	3000.00
2	4000.00
3	4250.00

Employee

EmployeeID	Ename	DeptID	Salary
1001	John	2	4000
1002	Anna	1	3500
1003	James	1	2500
1004	David	2	5000
1005	Mark	2	3000
1006	Steve	3	4500
1007	Alice	3	3500

```
SELECT DeptID, AVG(Salary)  
FROM Employee  
GROUP BY DeptID;
```

GROUP BY
Employee Table
using DeptID

DeptID	AVG(Salary)
1	3000.00
2	4000.00
3	4250.00

```
SELECT DeptID, AVG(Salary)  
FROM Employee  
GROUP BY DeptID  
HAVING AVG(Salary) > 3000;
```

HAVING

DeptID	AVG(Salary)
2	4000.00
3	4250.00

Select * from Employee;

	EMP_ID	NAME	DEPT_NAME	SALARY
▶	100	ABC	ENG	50000
	101	DEF	ENG	60000
	102	GHI	PS	50000
	103	JKL	PS	70000
	104	MNO	SALES	75000
	105	PQR	MKTG	70000
	106	STU	SALES	

Is this query correct?

```
SELECT DEPT_NAME,SUM(SALARY) FROM EMPLOYEE  
GROUP BY  
DEPT_NAME  
WHERE DEPT_NAME = 'ENG';
```

```
SELECT DEPT_NAME,SUM(SALARY) FROM EMPLOYEE  
GROUP BY  
DEPT_NAME  
HAVING DEPT NAME = 'ENG';
```

Select COUNT(*),COUNT(SALARY) FROM EMPLOYEE;
It would be

COUNT(*)	COUNT(SALARY)
7	6

Because COUNT(*) is not going to ignore the Nulls in the result
whereas COUNT(SALARY) is going to ignore the Nulls.

Remember : COUNT(*) is the only function which won't ignore Nulls. Other functions like SUM,AVG,MIN,MAX they ignore Nulls. What it means is in the previous query the salary value for a particular employee was NULL. So the query

EMPLOYEE_ID	FIRST_NAME	LAST_NAME		SALARY		MANAGER_ID		DEPARTMENT_ID
100	Steven	King	—	24000	—	0	—	90
101	Neena	Kochhar	—	17000	—	100	—	90
102	Lex	De Haan	—	17000	—	100	—	90
103	Alexander	Hunold	—	9000	—	102	—	60
104	Bruce	Ernst	—	6000	—	103	—	60
105	David	Austin	—	4800	—	103	—	60
106	Valli	Pataballa	—	4800	—	103	—	60
107	Diana	Lorentz	—	4200	—	103	—	60
108	Nancy	Greenberg	—	12000	—	101	—	100
—	—	—	—	—	—	—	—	—

```

SELECT manager_id, MIN(salary)
FROM employees
WHERE manager_id IS NOT NULL
GROUP BY manager_id
ORDER BY MIN(salary) DESC ;

```

manager_id have arranged in a group
and minimum salaries have arranged in
descending order

MANAGER_ID	SALARY
0	24000
100	17000
100	17000
102	9000
103	6000
103	4800
103	4800
103	4200

manager_id	MIN(salary)
0	24000
102	9000
205	8300
145	7000
146	7000
108	6900
147	6200
149	6200
148	6100

Sno	Where Clause	Having Clause
1	The WHERE clause specifies the criteria which individual records must meet to be selected by a query. It can be used without the GROUP BY clause	The HAVING clause cannot be used without the GROUP BY clause.
2	The WHERE clause selects rows before grouping.	The HAVING clause selects rows after grouping.
3	The WHERE clause cannot contain aggregate functions	The HAVING clause can contain aggregate functions.
4	WHERE clause is used to impose condition on SELECT statement as well as single row function and is used before GROUP BY clause	HAVING clause is used to impose condition on GROUP Function and is used after GROUP BY clause in the query
5	SELECT Column,AVG(Column_nmae)FROM Table_name WHERE Column > value GROUP BY Column_nmae	SELECT Columnq, AVG(Column_nmae)FROM Table_name WHERE Column > value GROUP BY Column_nmae Having column_name>or<value

Set Operators

01

UNION ALL

- `SELECT order_id FROM sales UNION ALL SELECT order_id FROM sales_history;`

02

UNION

- `SELECT order_id FROM sales UNION SELECT order_id FROM sales_history;`

03

INTERSECT

- `SELECT order_id FROM sales INTERSECT SELECT order_id FROM sales_history;`

04

MINUS

- `SELECT order_id FROM sales MINUS SELECT order_id FROM sales_history;`

001.	Display all the information of the emp table.
SQL>	Select * from emp ;
002.	Display unique jobs from EMP table.
SQL>	SELECT DISTINCT Job FROM EMP GROUP BY Job ;
003.	List the details of the emps in asc order of their salaries.
SQL>	SELECT * FROM EMP ORDER BY Sal ASC ;
004.	List the details of the emps in asc order of the Deptnos and desc of Jobs.
SQL>	SELECT * FROM EMP ORDER BY Deptno ASC, Job DESC ;
005.	Display all the unique job groups in the descending order
SQL>	select unique job from emp order by job desc ;

007.	List the emps who joined before 1981.
SQL>	select * from emp where hiredate < '01-Jan-1981' ;
008.	List the Empno, Ename, Sal, Daily Sal of all Employees in the ASC order of AnnSal.
SQL>	SELECT Empno, Ename, sal, Sal/30 DailySal FROM Emp ORDER BY Sal*12 ;
009.	Display the empno , ename, job, hiredate, exp of all Mgrs
SQL>	select empno, ename, sal, months_between(sysdate,hiredate)/12 Exp from emp where job = 'MANAGER' ;
010.	List the empno, ename, sal, exp of all emps working for Mgr 7839.
SQL>	select empno, ename, sal, months_between(sysdate,hiredate)/12 Exp from emp B where Mgr = 7839 ;
011.	Display the details of the emps whose Comm. Is more than their sal.
SQL>	select * from emp where comm > sal ;